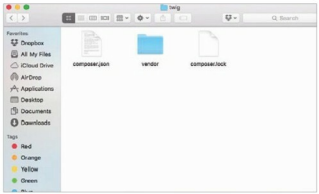

Figure 1: Installation of TWIG

composer on your CLI (terminal or cmd).

Installing TWIG

Now that you have successfully installed Composer on your machine, it's time to bring in the giant. Follow the steps given below to install and start using TWIG in your projects.

First, it is necessary to traverse into your project directory on your CLI to install TWIG.

Step 1: Execute the command:

```
composer require "twig/twig:~1.0"
```

TWIG is now installed on your project. When this has been executed, you will have two files and a folder in your project's folder, namely, *composer.json*, *composer.lock* and *vendor* (which contains TWIG). See Figure 1 above.

All you need now is to use TWIG's API to start implementing it in your project files.

So now, let's create a PHP API to render TWIG templates, create a PHP file named *index.php* and a new folder named *views*, with the following code in it:

```
<?php
require_once 'vendor/autoload.php';
    $loader = new Twig_Loader_Filesystem('views');
    $twig = new Twig_Environment($loader);
echo $twig->render('page.html', array('name' => 'Sameer'));
?>
```

Next, let's create a *page.html* file in the *views* folder, which contains the following lines of code:

```
<html>
    <head>
      <title>Welcome!</title>
    </head>
    <body>
      hello {{name}}, what are you doing here?
    </body>
</html>
```

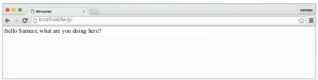

Figure 2: Output of page.html

Now open the browser and enter the URL *localhost/~project_folder~/*

The output should be something like what's shown in Figure 2.

So, what happened here? If you want to know how the initial '{{name}}' has changed to 'Sameer' here, you are in the right place. Now let's rewind a bit; in the file *index.php*, you included a file *autoload.php*, which is nothing more than an anchor file that links your project to TWIG's pre-built classes like *Twig_Loader_Filesystem* that you used in it.

So in that *index.php* file, you implemented TWIG's functions in the *views* folder, and then you loaded up a file named *page.html* passing an array to it (*'name' => 'Sameer'*). Now comes the magic trick; TWIG has three main signs of execution: *{{ }}*, *{% %}* and *{# #}*. When either of these has been detected, instantly, TWIG responds and gives out the proper output.

Each of these signs is assigned for a specific task.

{{ }} is used to display a variable.

{% %} is used to perform programmatic operations such as looping, including inheriting a parent document, etc.

{# #} is used to add comments to the code.

Our main focus will be on using programmatic operations –the *{% %}* operations.

Features available out-of-the-box

With its many capabilities, this lightweight framework extends the horizons of markup languages with its efficient features, which you get to use right out-of-the-box. Publishing constraints do not permit the explanation of all of these in one article.

Use of looping in TWIG: Every programmer wants this feature to reduce lines of code in repetitive conditions. TWIG happens to serve this purpose well. With its *{% for %}* statement, TWIG is capable of printing loop statements. As an example, let's change the array passed to *page.html* via *index.php* in the snippet previously given; so the changed file is as follows:

```
    <?php
            require_once 'vendor/autoload.php';
        $loader = new Twig_Loader_Filesystem('views');
        $twig = new Twig_Environment($loader);
              echo $twig->render('page.html', array('name' =>
'Sameer', 'users'=> array)
            array('name'=>'Rajan', 'age'=>'16'),
            array('name'=>'Rhythem', 'age'=>'19'),
            array('name'=>'Fazil', 'age'=>'21')
)
```